Emergency Training Simulator POC

Markdown documentation PDF copy

Thin Unreal Client Contract

Source: unreal/contract/client-contract.md

Markdown stays editable. This PDF is the reading/share copy.

Boundary: this project is still a POC, not operational guidance.

Thin Unreal Client Contract

Status: contract only. No Unreal scene or gameplay implementation.

The future Unreal client should be thin. It sends user actions to the backend and displays the state it gets back.

The backend owns the scenario. Unreal does not.

WebSocket endpoint

```
ws://{backend-host}/ws/sessions/{session_id}
```

For local development:

```
ws://127.0.0.1:8000/ws/sessions/{session_id}
```

Connection lifecycle

1. The client opens the WebSocket.
2. The client sends start_session.
3. The backend sends session_started with the starting state.
4. The client sends one ordered trainee_action.
5. The backend sends scenario_state_update with the new state and validation result.
6. The client repeats one action at a time.
7. If the state is terminal, the backend sends scenario_ended and closes the session.
8. If a message is malformed or unsupported, the backend sends error. The client should stop sending actions and start a new session if needed.

Client responsibilities

- Keep the WebSocket lifecycle simple and observable.
- Generate unique message IDs and event IDs for outbound trainee actions.
- Increment sequence_number for each outbound action.
- Display or cache backend state as received.
- Surface validation warnings/errors as feedback from the backend.
- Treat scenario_ended as the final state for the session.

Backend responsibilities

- Load the scenario.
- Apply deterministic transition rules.
- Check actions against POC rules.
- Produce the backend-owned scenario state.
- Write JSONL session logs.
- Decide when the scenario has ended.

Explicit non-responsibilities for Unreal

Unreal should not own:

- Scenario authority or transition rules
- Validation or scoring logic
- Medical, clinical, EMT, or firefighter protocol authority
- Hub-and-spoke networking
- Projection deployment
- Multimodal sensing
- Generated asset pipelines

- Gameplay logic beyond translating user interaction into contract-shaped actions later

Example messages

- start_session.message.json
- scenario_state_update.message.json
- validation_error_state_update.message.json
- scenario_ended.message.json